

CO2_Booster_Report_CF

March 10, 2026

1 CO -Booster-Kälteanlage — Expander-Potentialanalyse

Erstellt für: Christof Fischer GmbH (Kälte Fischer) Ansprechpartner: Benaja Löttsch, Simon Winter
Erstellt von: Ecalia GmbH — Michael Ilewicz, Valentin Koch

1.1 Inhaltsverzeichnis

1. Zusammenfassung
2. Randbedingungen & Betriebspunkte
3. Modellbeschreibung & Validierung
4. Ergebnisse — COP-Vergleich
5. Thermodynamische Diagramme (log p-h)
6. Energiekostenanalyse
7. COP-Sensitivität über Zwischendruck (mit TK-Stufe)
8. Empfehlung & nächste Schritte

```
[39]: # Setup
import sys, os, warnings
warnings.filterwarnings('ignore')
sys.path.insert(0, os.path.abspath('.'))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Markdown, HTML
from CoolProp.CoolProp import PropsSI as PSI
from copy import deepcopy

# Hide code cells in HTML/PDF export
display(HTML("""<style>
div.jp-Cell-inputWrapper { display: none !important; }
div.jp-OutputArea-prompt { display: none !important; }
</style>"""))

plt.rcParams.update({
    'figure.dpi': 150, 'savefig.dpi': 150,
    'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize': 10,
```

```

    'mathtext.fontset': 'cm',
})

from src.co2_booster import CO2BoosterFlashgasIWT, CO2BoosterParallel
from src.operating_points import (
    OP_SUMMER_NO_PV, OP_SUMMER_WITH_PV,
    OP_TRANSITION_NO_PV, OP_TRANSITION_WITH_PV,
    OP_WRG_NO_PV, OP_WRG_WITH_PV,
    OP_WINTER,
)
from src.plotting import (
    plot_logph_diagram, plot_logph_with_massflow, plot_logph_overlay,
    _draw_logph_background, _draw_model_on_axis, _annotate_state_points,
    _deduplicate_state_points, _annotate_canonical, _COMPONENT_NAMES,
    ECALIA_BLUE, ECALIA_TEAL, ECALIA_RED, ECALIA_ORANGE,
)
from fluprodia import FluidPropertyDiagram

```

<IPython.core.display.HTML object>

1.2 1. Zusammenfassung

Thermodynamische Simulation einer CO₂-Booster-Kälteanlage ($\dot{Q} = 98$ kW NK-Stufe) zur Quantifizierung des Energieeinsparpotentials durch Scroll-Expander. $\dot{m}_s$ pro Betriebspunkt an BITZER kalibriert → Baseline exakt. Alle Expander-Konfigurationen bei $p_{\text{med}} = 43$ bar mit IWT vor Expander für fairen Vergleich mit dem PV.

1.2.1 Kernergebnisse (Hochsommer, transkritisch, Baseline COP = 1.56 = BITZER)

Konfiguration	p_{med}	COP-Verbesserung	Jährl. Ersparnis
MD-Expander	43 bar	+11 %	ca. 3 400 €/a
HD-Expander	43 bar	+15 %	ca. 4 300 €/a
HD + MD Kombination	43 bar	+26 %	ca. 7 200 €/a
Parallelverdichter (BITZER Ref.) [†]	43 bar	+20 %	ca. 5 500 €/a [†]
PV + HD-Expander [†]	43 bar	+39 %	ca. 9 300 €/a [†]

[†] PV: Im Winter nicht betriebsfähig, in der Übergangsjahreszeit am unteren Regelungslimit (30 Hz). Flashgas-Verfügbarkeit unklar → PV-Ersparnisse vermutlich überschätzt. Supermärkte mit Dach-PV haben im Sommer niedrigere Stromkosten, was den PV-Vorteil weiter relativiert (s. Kap. 6).

Empfehlung: MD-Expander als Einstieg — stabiles Druckgefälle (43 → 26 bar), ganzjähriger Betrieb in 8 760 h/a, auch im Winter bei vollem Strompreis.

1.3 2. Randbedingungen & Betriebspunkte

Betriebspunkte von CF (BITZER-Software), 24/7/365 = 8 760 h/a.

Betriebspunkt	Betriebsart	t	p_high	t_GC/kond	p_med	COP_BITZER	Stl./a
Hochsommer	Transkritisch	−10 °C	93.7 bar	38 °C	38 bar	1.56	800
Übergangsjahreszeit	Subkritisch	−10 °C	65.3 bar*	27 °C	38 bar	2.97	3 160
WRG-Modus	Transkritisch	−10 °C	85.0 bar	25 °C	38 bar	2.36	2 000
Winter	Subkritisch	−10 °C	47.9 bar*	17 °C	38 bar	4.54	2 800

* Kondensationsdruck bei subkritischem Betrieb

1.3.1 Stundenverteilung (eigene Abschätzung)

Temperaturbereich	Betriebspunkt	Stunden
> 30 °C	Hochsommer	800
15–30 °C	Übergangsjahreszeit	3 160
5–15 °C	WRG-Modus	2 000
< 5 °C	Winter	2 800

Basis: typische Temperatur-Häufigkeitsverteilung süddeutscher Raum. Für ein konkretes Projekt sollte die Verteilung aus TRY-Daten oder der Gebäudeleittechnik abgeleitet werden.

1.3.2 PV-Betriebspunkte

Betriebspunkt	COP_BITZER (PV)	p_med (PV)	PV-Leistung
Hochsommer	1.85	43 bar	14.68 kW (4JTE-15K, 62 Hz)
Übergangsjahreszeit	3.43	43 bar	1.66 kW (2MTE-5K, 30 Hz)
WRG-Modus	2.48	38 bar	6.40 kW (2KTE-7K, 57 Hz)
Winter	—	—	PV nicht betriebsfähig

1.3.3 Simulationsparameter

Parameter	Wert
NK-Kompressor _s	pro OP kalibriert (0.66–0.69), s. Kap. 3
PV-Kompressor _s	= NK (nicht separat kalibrierbar, s. Kap. 3)
Expander _exp	0.70 (konservativ)
Strompreis (Netzbezug)	0.22 €/kWh
Strompreis (Dach-PV)	saisonabhängig: 0.16–0.22 €/kWh

1.3.4 Annahmen

- Betriebspunkte zeitlich konstant (keine Teillast, keine Übergänge).
- Vergleich bei gleicher Kälteleistung $Q = 98 \text{ kW}$ — Einsparung rein aus COP-Unterschied.
- Alle Expander-Konfigurationen bei $p_{\text{med}} = 43 \text{ bar}$ (= PV-Mitteldruck) mit IWT vor Expander für fairen Vergleich.
- PV kann nur 30–70 Hz regeln; im Winter nicht betriebsfähig, in der Übergangszeit durchgängig bei 30Hz.
- Reine NK-Betrachtung (TK-Stufe nicht modelliert, s. aber COP-Sensitivität Kap. 7).

```
[40]: # Modelle berechnen - kalibrierte _s pro Betriebspunkt (NK-Kompressor)
ETA_S_CALIBRATED = {
    "Hochsommer": 0.691,    # 3.5 - BITZER COP 1.56
    "Übergang": 0.684,    # 2.5 - BITZER COP 2.97
    "WRG-Modus": 0.679,    # 3.2 - BITZER COP 2.36
    "Winter": 0.655,      # 2.0 - BITZER COP 4.54
}

ETA_E = 0.70
HOURS_PER_YEAR = {"Hochsommer": 800, "Übergang": 3160, "WRG-Modus": 2000,
    ↪ "Winter": 2800}

# Strompreise
ELECTRICITY_PRICE_FLAT = 0.22 # €/kWh - reiner Netzbezug

# Szenario "Supermarkt mit Dach-PV": PV-Eigenverbrauch senkt effektive Kosten
    ↪ im Sommer
# Annahme: PV-LCOE 0.07 €/kWh; Eigenverbrauchsanteil saisonabhängig
ELECTRICITY_PRICE_PER_OP = {
    "Hochsommer": 0.16, # ~40 % PV-Eigenverbrauch
    "Übergang": 0.18, # ~25 % PV
    "WRG-Modus": 0.20, # ~15 % PV
    "Winter": 0.22, # kaum PV → voller Netzbezug
}

# Alle Expander-Konfigurationen bei p_med = 43 bar und IWT vor Expander
    ↪ (before_exp)
# für einen fairen Vergleich mit dem PV, der ebenfalls bei 43 bar arbeitet.
IWT_POSITION = "before_exp"

configs = {
    "Baseline (V+V)": {"cls": CO2BoosterFlashgasIWT, "kwargs": {}},
    "MD-Exp": {"cls": CO2BoosterFlashgasIWT,
        "kwargs": {"expansion_device_fg": "expander",
            "iwt_position": IWT_POSITION},
        "p_med_override": 43.0},
    "HD-Exp": {"cls": CO2BoosterFlashgasIWT,
        "kwargs": {"expansion_device_hp": "expander",
```

```

        "iwt_position": IWT_POSITION},
        "p_med_override": 43.0},
    "HD+MD Exp": {"cls": CO2BoosterFlashgasIWT,
                  "kwargs": {"expansion_device_hp": "expander",
                             "expansion_device_fg": "expander",
                             "iwt_position": IWT_POSITION},
                  "p_med_override": 43.0},
    "PV (Parallel)": {"cls": CO2BoosterParallel, "kwargs": {}},
    "PV + HD-Exp": {"cls": CO2BoosterParallel,
                    "kwargs": {"expansion_device_hp": "expander"}},
}

# WICHTIG: PV-Konfigurationen verwenden jeweils die PV-spezifischen
↳ Betriebspunkte
# von BITZER (anderer p_medium, anderer Kompressor). Ohne PV → flashgas-OP
↳ (p_med=38 bar).
test_ops = {
    "Hochsommer": {"flashgas": OP_SUMMER_NO_PV, "pv": OP_SUMMER_WITH_PV},
    "Übergang": {"flashgas": OP_TRANSITION_NO_PV, "pv":
↳ OP_TRANSITION_WITH_PV},
    "WRG-Modus": {"flashgas": OP_WRG_NO_PV, "pv": OP_WRG_WITH_PV},
    "Winter": {"flashgas": OP_WINTER, "pv": OP_WINTER},
}

# PV kann im Winter nicht betrieben werden (laut CF: 0 % Regelung).
PV_UNAVAILABLE_OPS = {"Winter"}

all_results = {}
all_models = {}

for cfg_name, cfg_def in configs.items():
    all_results[cfg_name] = {}
    for op_name, ops in test_ops.items():
        cls = cfg_def["cls"]
        is_pv = (cls == CO2BoosterParallel)

        if is_pv and op_name in PV_UNAVAILABLE_OPS:
            all_results[cfg_name][op_name] = None
            continue

        op = ops["pv"] if is_pv else ops["flashgas"]

        p_med_ovr = cfg_def.get("p_med_override")
        if p_med_ovr:
            op = deepcopy(op)
            op.p_medium = p_med_ovr

```

```

eta_s = ETA_S_CALIBRATED[op_name]

try:
    kw = {**cfg_def["kwargs"],
          "compressor_efficiency": eta_s,
          "expander_efficiency": ETA_E}
    model = cls(**kw)
    model.setup_network(iterinfo=False)
    model.set_boundary_conditions(op=op)
    model.solve()
    if model.converged:
        cop = model.calculate_cop_cooling()
        power = model.get_power_breakdown()
        P_comp = sum(d['P_kW'] for d in power.values() if d['type'] ==
↪ 'compressor')
        P_exp = sum(d['P_kW'] for d in power.values() if d['type'] ==
↪ 'expander')
        all_results[cfg_name][op_name] = {
            "cop": cop, "P_comp": P_comp, "P_exp": P_exp, "P_net":
↪ P_comp + P_exp,
        }
        all_models[(cfg_name, op_name)] = model
    except Exception:
        all_results[cfg_name][op_name] = None

```

1.4 3. Modellvalidierung

η_s wurde pro Betriebspunkt an BITZER-Baseline kalibriert (NK, $p_{med} = 38$ bar) → Baseline-COPs stimmen exakt überein.

Expander-Konfigurationen laufen bei $p_{med} = 43$ bar (= PV-Mitteldruck) mit IWT vor Expander (before_exp-Topologie) für einen fairen Vergleich. η_s wurde bei 38 bar kalibriert → bei 43 bar verschobenes Druckverhältnis, aber Effekt auf relative Vergleiche gering.

Für PV-Konfigurationen wird derselbe η_s verwendet. Systematische Abweichungen entstehen durch den anderen Kompressor (PV-spezifisch). Die PV-Abweichungen (s. Tabelle) beeinflussen den relativen Vergleich Expander vs. PV nur geringfügig.

```

[41]: # --- Kalibrierung & Validierung ---
bitzer_cops = {"Hochsommer": 1.56, "Übergang": 2.97, "WRG-Modus": 2.36,
↪ "Winter": 4.54}
bitzer_cops_pv = {"Hochsommer": 1.85, "Übergang": 3.43, "WRG-Modus": 2.48}

# Baseline-Validierung (kalibrierte  $\eta_s$ )
val_rows = []
for op_name, cop_b in bitzer_cops.items():
    r = all_results["Baseline (V+V)"].get(op_name)

```

```

cop_t = r["cop"] if r else None
delta = (cop_t - cop_b) / cop_b * 100 if cop_t else None
val_rows.append({
    "Betriebspunkt": op_name,
    "_s (NK)": f"{ETA_S_CALIBRATED[op_name]:.3f}",
    "COP BITZER": f"{cop_b:.2f}",
    "COP TESPy": f"{cop_t:.2f}" if cop_t else "-",
    "Δ [%]": f"{delta:+.2f}" if delta is not None else "-",
})

df_val = pd.DataFrame(val_rows)
display(df_val.style.set_caption("Baseline-Validierung: kalibrierter _s pro_
↳Betriebspunkt").hide(axis='index'))

# PV-Validierung (gleicher _s wie Baseline - nicht separat kalibriert)
val_rows_pv = []
for op_name, cop_b in bitzer_cops_pv.items():
    r = all_results["PV (Parallel)"].get(op_name)
    cop_t = r["cop"] if r else None
    delta = (cop_t - cop_b) / cop_b * 100 if cop_t else None
    val_rows_pv.append({
        "Betriebspunkt": op_name,
        "_s (NK)": f"{ETA_S_CALIBRATED[op_name]:.3f}",
        "COP BITZER (PV)": f"{cop_b:.2f}",
        "COP TESPy (PV)": f"{cop_t:.2f}" if cop_t else "-",
        "Δ [%]": f"{delta:+.1f}" if delta is not None else "-",
    })

df_val_pv = pd.DataFrame(val_rows_pv)
display(df_val_pv.style.set_caption("PV-Validierung: gleicher _s wie_
↳Baseline").hide(axis='index'))

```

<pandas.io.formats.style.Styler at 0x2d33e78cc80>

<pandas.io.formats.style.Styler at 0x2d32c5e5340>

1.5 4. COP-Vergleich — Alle Konfigurationen

```

[42]: # COP-Tabelle
baseline_name = "Baseline (V+V)"
cop_rows = []
for cfg_name in configs:
    is_pv = configs[cfg_name]["cls"] == CO2BoosterParallel
    label = cfg_name + ("†" if is_pv else "")
    row = {"Konfiguration": label}
    for op_name in test_ops:

```

```

r = all_results[cfg_name].get(op_name)
r_base = all_results[baseline_name].get(op_name)
if r and r_base:
    cop = r["cop"]
    delta = (cop - r_base["cop"]) / r_base["cop"] * 100
    row[f"COP {op_name}"] = f"{cop:.2f}"
    row[f"Δ {op_name} [%]" = f"{delta:+.1f}"
else:
    row[f"COP {op_name}"] = "-"
    row[f"Δ {op_name} [%]" = "-"
cop_rows.append(row)

df_cop = pd.DataFrame(cop_rows)
display(df_cop.style.set_caption("COP-Vergleich: Alle Konfigurationen ×
↳Betriebspunkte").hide(axis='index'))
display(Markdown("† PV: Thermodynamisches Maximum bei voller Verfügbarkeit↳
↳(Einschränkungen s. Kap. 1)."))

```

<pandas.io.formats.style.Styler at 0x2d33e538ec0>

† PV: Thermodynamisches Maximum bei voller Verfügbarkeit (Einschränkungen s. Kap. 1).

```

[43]: # COP-Balkendiagramm (Hochsommer)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5.5))
cfg_names = list(configs.keys())
colors_all = [ECALIA_BLUE, ECALIA_TEAL, '#5fb88a', ECALIA_ORANGE, ECALIA_RED,
↳'#666666', '#333333']
# Sicherstellen, dass genug Farben da sind
while len(colors_all) < len(cfg_names):
    colors_all.append('#888888')

cops_hs = []
for c in cfg_names:
    r = all_results[c].get("Hochsommer")
    cops_hs.append(r["cop"] if r else 0)

bars = ax1.barh(range(len(cfg_names)), cops_hs, color=colors_all[:
↳len(cfg_names)])
for bar, val in zip(bars, cops_hs):
    if val > 0:
        ax1.text(bar.get_width()+0.02, bar.get_y()+bar.get_height()/2,
            f"{val:.2f}", va='center', fontsize=9)
ax1.set_yticks(range(len(cfg_names)))
ax1.set_yticklabels(cfg_names, fontsize=8)
ax1.set_xlabel("COP")
ax1.set_title("COP - Hochsommer")
ax1.invert_yaxis()
ax1.margins(x=0.1) # mehr Platz am unteren Balken

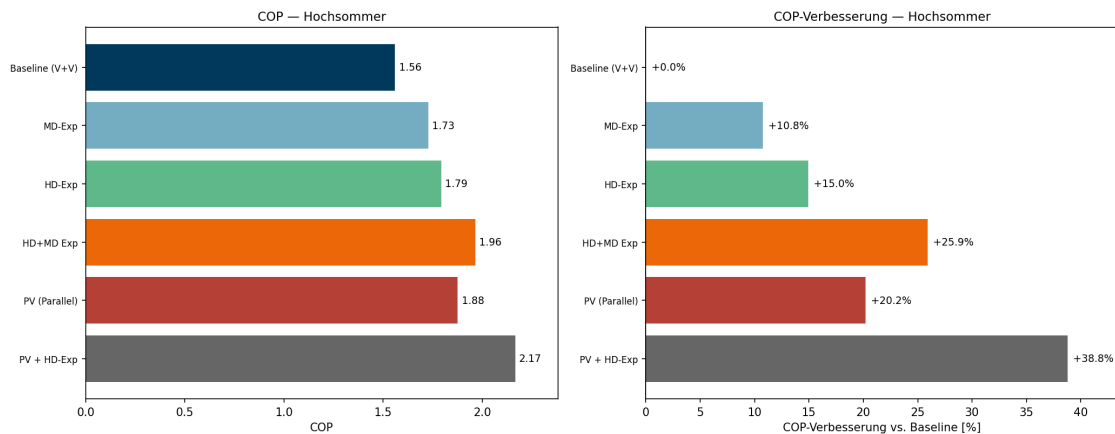
```



```

baseline_hs = all_results["Baseline (V+V)"]["Hochsommer"]["cop"]
deltas = [(c - baseline_hs)/baseline_hs*100 if c > 0 else 0 for c in cops_hs]
bars2 = ax2.barh(range(len(cfg_names)), deltas, color=colors_all[:
    ↪len(cfg_names)])
for bar, val in zip(bars2, deltas):
    ax2.text(max(bar.get_width(),0)+0.5, bar.get_y()+bar.get_height()/2,
        f"{val:+.1f}%", va='center', fontsize=9)
ax2.set_yticks(range(len(cfg_names)))
ax2.set_yticklabels(cfg_names, fontsize=8)
ax2.set_xlabel("COP-Verbesserung vs. Baseline [%]")
ax2.set_title("COP-Verbesserung - Hochsommer")
ax2.invert_yaxis()
ax2.margins(x=0.12)
ax2.axvline(x=0, color='black', linewidth=0.5)
fig.tight_layout()
plt.show()

```



1.6 5. Thermodynamische Diagramme

$\log(p)$ - h Vergleich: Baseline (38 bar) vs. Vergleichskonfiguration (43 bar, Hochsommer). Expander-Konfigurationen nutzen die `before_exp`-Topologie (IWT vor Expander): Das Flashgas wird im IWT auf Mitteldruckniveau so weit überhitzt, dass es nach Expansion am Saugdruckniveau dieselbe Überhitzung aufweist wie die Baseline-Saugseite — maximale Unterkühlung, kein Kondensationsrisiko im Expander.

Hinweis IWT-Fläche: Sollte der IWT für die benötigte Überhitzung zu groß werden, kann alternativ nach Expansion wärmegetauscht werden (`after_exp`-Topologie). Dann wird das Flashgas bei teils flüssiger Phase im IWT erwärmt — hydraulisch einfacher, aber geringfügig niedrigerer COP (ca. -0.5%).

```

[44]: # log(p)-h Overlay mit kanonisch nummerierten Zustandspunkten

overlay_configs = [
    ("Baseline vs MD-Exp", "Baseline (V+V)", "MD-Exp",          ECALIA_BLUE,
    ↪ECALIA_ORANGE),
    ("Baseline vs PV",      "Baseline (V+V)", "PV (Parallel)", ECALIA_BLUE,
    ↪ECALIA_TEAL),
    ("Baseline vs HD+MD",   "Baseline (V+V)", "HD+MD Exp",      ECALIA_BLUE,
    ↪ECALIA_RED),
]

fig, axes = plt.subplots(1, 3, figsize=(20, 6))
panel_legends = {}

for idx, (title, cfg1, cfg2, color1, color2) in enumerate(overlay_configs):
    m1 = all_models.get((cfg1, "Hochsommer"))
    m2 = all_models.get((cfg2, "Hochsommer"))
    if m1 is None or m2 is None:
        axes[idx].text(0.5, 0.5, f"Modell nicht verfügbar:\n{cfg2}",
                        ha='center', va='center', transform=axes[idx].transAxes)
        continue

    diagram = FluidPropertyDiagram('CO2')
    diagram.set_unit_system(T='°C', p='bar', h='kJ/kg')

    h_vals, p_vals = [], []
    for mdl in [m1, m2]:
        for k, data in mdl.get_results().items():
            dp = diagram.calc_individual_isoline(**data)
            h_vals.extend(dp['h']); p_vals.extend(dp['p'])
        for st in mdl.get_state_points().values():
            h_vals.append(st['h']); p_vals.append(st['p'])

    ax = axes[idx]
    _draw_logph_background(diagram, fig, ax,
                           min(h_vals)-30, max(h_vals)+30,
                           min(p_vals)*0.7, max(p_vals)*1.3)
    _draw_model_on_axis(m1, diagram, ax, color1, linewidth=1.5, label=cfg1)
    _draw_model_on_axis(m2, diagram, ax, color2, linewidth=2.5, label=cfg2)

    # Kanonisch nummerierte Zustandspunkte (konsistent über alle Topologien)
    canon_tuples = _annotate_canonical(m2, ax, fontsize=6)
    panel_legends[title] = (cfg2, canon_tuples)

    ax.set_title(title, fontsize=10)
    ax.legend(fontsize=8)

```

```

fig.suptitle(r"$\log(p)$-$h$ Vergleich - Hochsommer", fontsize=13)
fig.tight_layout()
plt.show()

# Zustandspunkt-Tabelle im Wide-Format (Konfigurationen als Spaltengruppen)
# Beschreibungsspalte aus dem ersten Panel übernehmen (identisch für alle)
first_cfg = list(panel_legends.keys())[0]
desc_map = {num: desc for num, desc, _, _ in panel_legends[first_cfg][1]}

wide_parts = []
cfg_order = []

for panel_title, (cfg2_name, canon_tuples) in panel_legends.items():
    cfg_order.append(cfg2_name)
    rows = {}
    for num, desc, h_val, p_val in canon_tuples:
        rows[num] = {"h [kJ/kg]": h_val, "p [bar]": p_val}
    sub = pd.DataFrame.from_dict(rows, orient="index")
    sub.index.name = "Nr."
    sub.columns = pd.MultiIndex.from_product([[cfg2_name], sub.columns])
    wide_parts.append(sub)

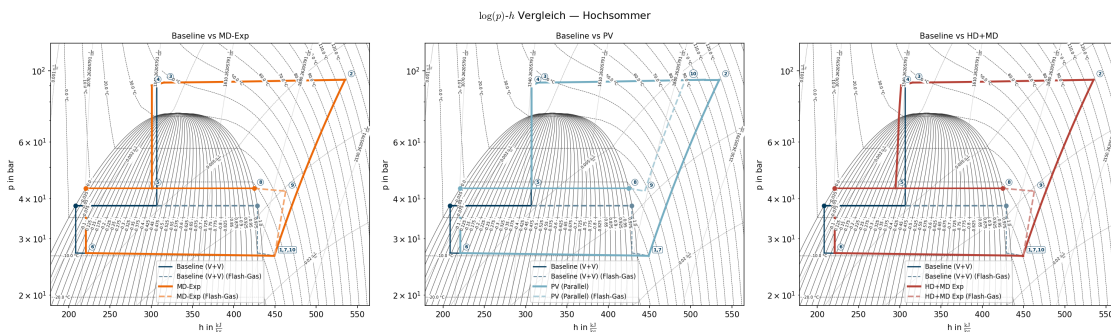
df_wide = pd.concat(wide_parts, axis=1).sort_index()

# Beschreibung als erste Spalte einfügen
df_wide.insert(0, "Zustandspunkt", [desc_map.get(i, "") for i in df_wide.index])

# Format: 2 Nachkommastellen, fehlende Punkte als "-"
df_wide = df_wide.map(lambda x: f"{x:.2f}" if isinstance(x, (int, float)) else
    ↪(x if isinstance(x, str) else "-"))

display(df_wide.style
    .set_caption("Zustandspunkte - Vergleichskonfigurationen (Hochsommer)")
)

```



<pandas.io.formats.style.Styler at 0x2d33e583500>

1.7 6. Energiekostenanalyse

Zwei Szenarien: (A) Netzbezug 0.22 €/kWh, (B) Supermarkt mit Dach-PV (0.16–0.22 €/kWh saisonabhängig). Dach-PV senkt Sommerkosten → relativiert den finanziellen Vorteil von Konfigurationen, die primär im Sommer sparen.

[45]: *# Energiekostenanalyse - zwei Szenarien: Flat-Rate vs. Dach-PV*

```
rows_energy = []
for cfg_name in configs:
    if cfg_name == "Baseline (V+V)":
        continue
    E_base, E_cfg = 0.0, 0.0
    savings_by_op = {}
    cost_flat, cost_pv = 0.0, 0.0
    pv_note = ""
    is_pv = configs[cfg_name]["cls"] == CO2BoosterParallel

    for op_name, hours in HOURS_PER_YEAR.items():
        r_base = all_results["Baseline (V+V)"].get(op_name)
        r_cfg = all_results[cfg_name].get(op_name)
        if r_base and r_cfg:
            dP = r_base["P_net"] - r_cfg["P_net"] # kW gespart
            dE = dP * hours # kWh gespart
            E_base += r_base["P_net"] * hours
            E_cfg += r_cfg["P_net"] * hours
            savings_by_op[op_name] = dE
            cost_flat += dE * ELECTRICITY_PRICE_FLAT
            cost_pv += dE * ELECTRICITY_PRICE_PER_OP[op_name]
        elif r_base:
            E_base += r_base["P_net"] * hours
            E_cfg += r_base["P_net"] * hours
            savings_by_op[op_name] = 0.0
            if is_pv and op_name in PV_UNAVAILABLE_OPS:
                pv_note = "†"

    delta_E = E_base - E_cfg

    rows_energy.append({
        "Konfiguration": cfg_name + pv_note,
        "ΔE [MWh/a]": f"{delta_E / 1000:.1f}",
        "Netzbezug [€/a]": f"{cost_flat:,.0f}",
        "Mit Dach-PV [€/a]": f"{cost_pv:,.0f}",
        "_delta_E": delta_E, "_cost_flat": cost_flat, "_cost_pv": cost_pv,
```

```

        "_savings_op": savings_by_op,
    })

df_energy = pd.DataFrame(rows_energy)
display_cols = [c for c in df_energy.columns if not c.startswith("_")]
display(df_energy[display_cols].style
        .set_caption("Jährliche Energieeinsparung - Flat-Rate (0.22 €/kWh) vs.
↳ Dach-PV (0.16-0.22 €/kWh)")
        .hide(axis='index'))

display(Markdown(
    "† PV: Im Winter Baseline-Verbrauch, Übergang 30 Hz. "
    "Dach-PV senkt effektive Stromkosten im Sommer → Expander profitiert, "
    "da er ganzjährig spart (auch im Winter bei vollem Netzbezugspreis).")
))

```

<pandas.io.formats.style.Styler at 0x2d32cd5bf20>

† PV: Im Winter Baseline-Verbrauch, Übergang 30 Hz. Dach-PV senkt effektive Stromkosten im Sommer → Expander profitiert, da er ganzjährig spart (auch im Winter bei vollem Netzbezugspreis).

```

[46]: # Energiekostenvisualisierung - Flat vs. Dach-PV
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
cfg_labels = df_energy["Konfiguration"].tolist()
op_colors = {"Hochsommer": ECALIA_RED, "Übergang": ECALIA_ORANGE,
             "WRG-Modus": ECALIA_TEAL, "Winter": ECALIA_BLUE}

# Links: Stacked bar - Einsparung nach Betriebspunkt (kWh, preisunabhängig)
bottom = np.zeros(len(cfg_labels))
for op_name, op_color in op_colors.items():
    vals = [row["_savings_op"].get(op_name, 0) / 1000 for _, row in df_energy.
↳ iterrows()]
    ax1.bar(range(len(cfg_labels)), vals, bottom=bottom, color=op_color,
            label=f"{op_name} ({HOURS_PER_YEAR[op_name]} h/a)",
↳ edgecolor='white', linewidth=0.5)
    bottom += np.array(vals)
for i, (_, row) in enumerate(df_energy.iterrows()):
    ax1.text(i, bottom[i] + 0.3, f"{row['ΔE [MWh/a]']} MWh",
            ha='center', fontsize=9)
ax1.set_xticks(range(len(cfg_labels)))
ax1.set_xticklabels(cfg_labels, rotation=30, ha='right', fontsize=7)
ax1.set_ylabel("Einsparung [MWh/a]")
ax1.set_title("Jährliche Stromeinsparung")
ax1.legend(fontsize=7)
ax1.grid(axis='y', alpha=0.3)

# Rechts: Kosteneinsparung - Flat vs. Dach-PV (gruppierte Balken)

```

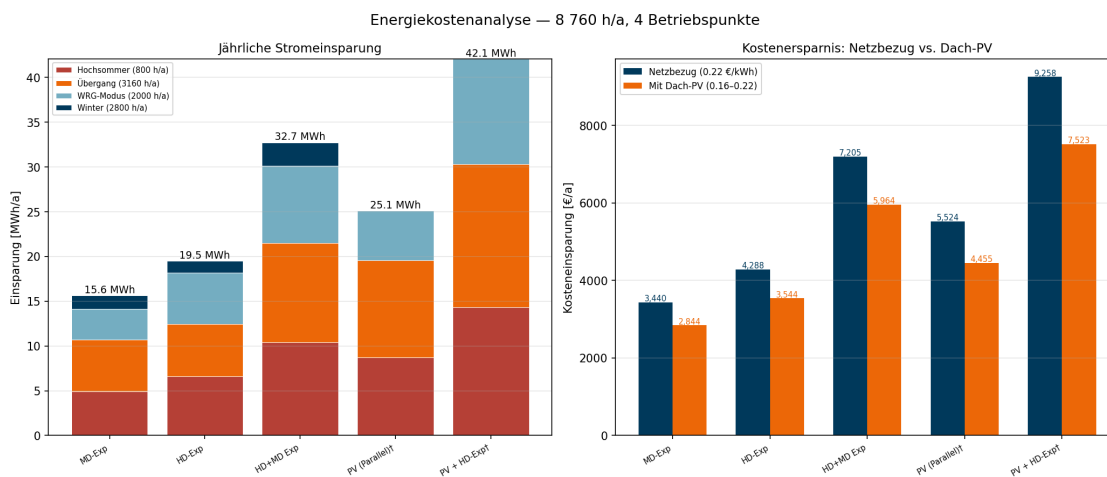
```

x = np.arange(len(cfg_labels))
width = 0.35
cost_flat = [row["_cost_flat"] for _, row in df_energy.iterrows()]
cost_pv = [row["_cost_pv"] for _, row in df_energy.iterrows()]

bars_flat = ax2.bar(x - width/2, cost_flat, width, color=ECALIA_BLUE,
    ↪label="Netzbezug (0.22 €/kWh)")
bars_pv = ax2.bar(x + width/2, cost_pv, width, color=ECALIA_ORANGE,
    ↪label="Mit Dach-PV (0.16-0.22)")
for bar, val in zip(bars_flat, cost_flat):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 30,
        f"{val:,.0f}", ha='center', fontsize=7, color=ECALIA_BLUE)
for bar, val in zip(bars_pv, cost_pv):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 30,
        f"{val:,.0f}", ha='center', fontsize=7, color=ECALIA_ORANGE)
ax2.set_xticks(x)
ax2.set_xticklabels(cfg_labels, rotation=30, ha='right', fontsize=7)
ax2.set_ylabel("Kosteneinsparung [€/a]")
ax2.set_title("Kostensparnis: Netzbezug vs. Dach-PV")
ax2.legend(fontsize=8)
ax2.grid(axis='y', alpha=0.3)

fig.suptitle("Energiekostenanalyse - 8 760 h/a, 4 Betriebspunkte", fontsize=13)
fig.tight_layout()
plt.show()

```



1.8 7. COP-Sensitivität über Zwischendruck

p_medium-Sweep (30–48 bar, Hochsommer) für Baseline vs. MD-Expander (IWT vor Expander).
Zusätzlich: Effekt einer TK-Stufe (15 kW bei -30 °C) auf den Gesamt-COP.

```
[47]: # COP vs. p_medium Sweep (Hochsommer, mit und ohne TK-Stufe)
from copy import deepcopy

ETA_S_HS = ETA_S_CALIBRATED["Hochsommer"] # 0.691

p_med_range = np.arange(30, 49, 2.0) # bar
op_base = OP_SUMMER_NO_PV

# TK-Stufe Parameter (typisch: -30 °C, 15 kW)
T_evap_tk = -30.0 # °C
QO_TK = 15.0 # kW
p_evap_tk = PSI("P", "Q", 1, "T", 273.15 + T_evap_tk, "CO2") / 1e5

# Build result arrays
cop_baseline = np.full(len(p_med_range), np.nan)
cop_md_exp = np.full(len(p_med_range), np.nan)
cop_baseline_tk = np.full(len(p_med_range), np.nan)
cop_md_exp_tk = np.full(len(p_med_range), np.nan)

for i, p_med in enumerate(p_med_range):
    for cfg_key, cfg_kw, arr_nk, arr_tk in [
        ("baseline", {}, cop_baseline, cop_baseline_tk),
        ("md_exp", {"expansion_device_fg": "expander", "iwt_position": "IWT_POSITION"}, cop_md_exp, cop_md_exp_tk),
    ]:
        try:
            op = deepcopy(op_base)
            op.p_medium = float(p_med)
            mdl = CO2BoosterFlashgasIWT(compressor_efficiency=ETA_S_HS,
                                       expander_efficiency=ETA_E, **cfg_kw)
            mdl.setup_network(iterinfo=False)
            mdl.set_boundary_conditions(op=op)
            mdl.solve()
            if mdl.converged:
                cop_nk = mdl.calculate_cop_cooling()
                P_nk = sum(d['P_kW'] for d in mdl.get_power_breakdown().
                           values())
                arr_nk[i] = cop_nk

            # TK stage estimate: compressor p_evap_tk → p_med
            h_tk_in = PSI("H", "Q", 1, "T", 273.15 + T_evap_tk, "CO2") / 1e3
            s_tk_in = PSI("S", "Q", 1, "T", 273.15 + T_evap_tk, "CO2")
```

```

        hTk_out_s = PSI("H", "S", sTk_in, "P", float(p_med) * 1e5,
↪ "CO2") / 1e3
        hTk_out = hTk_in + (hTk_out_s - hTk_in) / ETA_S_HS
        hTk_liq = PSI("H", "Q", 0, "P", p_evapTk * 1e5, "CO2") / 1e3
        mTk_kg_s = Q0_TK / (hTk_in - hTk_liq)
        WTk = abs(mTk_kg_s) * (hTk_out - hTk_in)
        arrTk[i] = (op.Q0_NK + Q0_TK) / (abs(P_nk) + abs(WTk))
    except Exception:
        pass # stays NaN

# Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5.5))

ax1.plot(p_med_range, cop_baseline, 'o-', color=ECALIA_BLUE,
        label="Baseline (nur NK)", linewidth=2)
ax1.plot(p_med_range, cop_md_exp, 's-', color=ECALIA_ORANGE,
        label="MD-Expander (nur NK)", linewidth=2)
ax1.plot(p_med_range, cop_baselineTk, 'o--', color=ECALIA_BLUE,
        alpha=0.6, label=r"Baseline (NK + TK 15 kW @ $-30\,^{\circ}\text{C})")
ax1.plot(p_med_range, cop_md_expTk, 's--', color=ECALIA_ORANGE,
        alpha=0.6, label=r"MD-Exp (NK + TK 15 kW @ $-30\,^{\circ}\text{C})")
ax1.axvline(x=38, color='gray', linestyle=':', alpha=0.5,
        label=r"$p_{\mathrm{med}}$ CF ohne PV (38 bar)")
ax1.axvline(x=43, color='green', linestyle=':', alpha=0.5,
        label=r"$p_{\mathrm{med}}$ CF mit PV (43 bar)")
ax1.set_xlabel(r"Zwischendruck $p_{\mathrm{medium}}$ [bar]")
ax1.set_ylabel("COP")
ax1.set_title(r"COP vs. $p_{\mathrm{medium}}$ - Hochsommer")
ax1.legend(fontsize=7)
ax1.grid(alpha=0.3)

# COP improvement %
cop_impr = np.where((cop_baseline > 0) & np.isfinite(cop_baseline),
        (cop_md_exp - cop_baseline) / cop_baseline * 100, np.nan)
cop_imprTk = np.where((cop_baselineTk > 0) & np.isfinite(cop_baselineTk),
        (cop_md_expTk - cop_baselineTk) / cop_baselineTk *
↪ 100, np.nan)
ax2.plot(p_med_range, cop_impr, 's-', color=ECALIA_ORANGE, linewidth=2,
↪ label="NK only")
ax2.plot(p_med_range, cop_imprTk, 's--', color=ECALIA_ORANGE, alpha=0.6,
↪ label="NK + TK")
ax2.axvline(x=38, color='gray', linestyle=':', alpha=0.5,
        label=r"$p_{\mathrm{med}}$ CF ohne PV (38 bar)")
ax2.axvline(x=43, color='green', linestyle=':', alpha=0.5,
        label=r"$p_{\mathrm{med}}$ CF mit PV (43 bar)")
ax2.set_xlabel(r"Zwischendruck $p_{\mathrm{medium}}$ [bar]")
ax2.set_ylabel("COP-Verbesserung durch MD-Exp [%]")

```



```

ax2.set_title(r"Expander-Vorteil vs.  $p_{\text{medium}}$ ")
ax2.legend(fontsize=8)
ax2.grid(alpha=0.3)

fig.suptitle(r"Sensitivität:  $p_{\text{medium}}$  und TK-Stufe", fontsize=13)
fig.tight_layout()
plt.show()

```

